29.10.2026

Лекция 8. Соединения и операции над множествами

JOIN

(INNER) JOIN

Соединяет 2 таблицы между собой по условию. `INNER` обычно не пишут.

Вывести 10 последних заказов с данными клиента

```
SELECT o.order_id, c.last_name, c.first_name, o.status,
o.total_count
FROM orders o
JOIN customers c ON c.customer_id = o.customer.id
ORDER BY o.purchased_at DESC, o.order_id DESC
LIMIT 10;
```

Для каждоого доставленного заказа вывести фамилию клиеннта, название купленного товара и цену

```
SELECT c.last_name, o.order_id, p.name, oi.price
FROM customers c
JOIN orders 0 ON o.customer_id = c.customer_id
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p ON p.product_id = oi.product_id
WHERE o.status = 'delivered'
LIMIT 10;
```

LEFT JOIN

`LEFT JOIN` сохраняет все строки левой таблицы.

Особенности:

- Если пары нет, столюцы правой таблицы заполняются `NULL`
- `LEFT JOIN` сохраняет все строчки левой таблицы
- `COUNT(*)` посчитает строчки - даже те, что с `NULL`

Вывести число заказов у каждого клиента

```
SELECT c.customer_id, COUNT(o.order_id) AS orders_cnt
FROM customers c
LEFT JOIN orders o ON o.customer_od = c.customer_id
GROUP BY c.customer_id
ORDER BY orders_cnt DESC c.customer_id;
```

Антисоединение

Найти клиентов у которых нет заказов

```
SELECT c.customer_id
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL
ORDER BY c.customer_id;
```

1. Проверяем на `NULL` первичный ключ правой таблицы
2. Столбец, который сам может быть `NULL` не подходит

ON и WHERE в LEFT JOIN

Посчитать доставленные заказы каждого клиента

Неправильный вариант:

```
SELECT c.customer_id, COUNT(o.order_id)
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.status = 'delivered'
GROUP BY c.customer_id;
```

Правильный вариант:

```sql
SELECT c.customer_id, COUNT(o.order_id) as delivered_orders
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
AND o.status = 'delivered'
GROUP BY c.customer_id;
```

Для `INNER JOIN` разницы нет.

Порядок соединнений

Если соединяем много `LEFT JOIN`'ов, то и последний `JOIN` должен быть `LEFT JOIN`. Поэтому можно сперва делать `INNER JOIN`, а потом к нему делать `RIGHT JOIN`.

Размножение строк (fan-out)

```sql
SELECT SUM(o.total_amount)
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id;
```

Запрос выше не корректен:

1. Строка заказа повторяется по числу позиций, сумма завышена
2. Заказы без позиций пропали

Как бороться с fan-out:

- Суммировать значения из правой таблицы: `SUM(oi.price)`
- Считать заказы через `COUNT(DISTINCT o.order_id)`
- Сначала агрегировать вторую таблицу, потом соединять

RIGHT JOIN

То же что и `LEFT JOIN`, но для правой таблицы

FULL JOIN

```sql
SELECT
COUNT(*) FILTER(WHERE d.order_id IS NULL) AS
orders_without_delivery,
COUNT(*) FILTER(WHERE o.order_id IS NULL) AS
deliveries_without_order
FROM orders o
FULL JOIN deliveries d ON d.order_id = o.order_id;
```

CROSS JOIN

Это декартово произведение. Для больших таблиц может выполняться долго или даже упасть с ошибкой (!!).

USING

Заменяет `ON`: `USING(order_id)` вместо `ON a.order_id = b.order_id`.

- Сливает одноименные столбцы в 1
- Работает, когда имена столбцов совпадают
- Запись короче чем с `ON`, но не такая гибкая (например, нельзя написать `AND`)

Самосоединение

Найти пары заказов одного клиента, оформленные с интервалом меньше суток

```sql
SELECT o1.customer_id,
o1.order_id AS first_order,
o2.order_id AS second_order,,
o2.purchased_at - o1.purchased_at AS gap
FROM orders o1
JOIN orders o2 ON o2.customer_id = o1.customer_id
AND o2.order_id > o1.order_id
AND o2.purchased_at - o1.purchased_at < INTERVAL '1 day'
ORDER BY gap;
```

Скрытые ловышки соединений:

1. Соединение по неключевым столюцам дает случайные пары
2. Если `NULL` нужно считать равными: `a IS NOT DISTINCT FROM b`
3. Цена позиции фиксируется на момент продажи и может отличаться от текущей
4. PostgreSQL не создает индекс на внешний ключ автоматически
5. Строки с `NULL` в ключе не соединяются

a	b	a = b	a IS NOT DISTINCT FROM b
1	1	TRUE	TRUE
1	2	FALSE	FALSE
1	NULL	UNKNOWN	FALSE
NULL	NULL	UNKNOWN	TRUE

EXISTS, коррелированные подзапросы, NOT IN

EXISTS

Проверяет факт существования каких-то данных. Его можно использовать для проверки наличия данных в подзапросе.

Найти курьеров, у которых есть хотя бы 1 доставка

```
SELECT c.courier_id, c.last_name
FROM couriers c
WHERE EXISTS (
    SELECT 1 FROM deliveries d
    WHERE d.courier_id = c.courier_id
);
```

Содержание подзапроса выше не имело значение, поэтому пишем `SELECT 1`.

NOT EXISTS

Истинен, если подзапрос не вернул ни 1 строки.

Найти заказы без доставки

```
SELECT o.order_id, o.status
FROM orders o
WHERE NOT EXISTS (
     SELECT 1 FROM deliveries d
     WHERE d.order_id = o.order_id
);
```

EXISTS и JOIN

- `EXISTS` не размножает строки
- `EXISTS` безопасен к `NULL`

Коррелированный подзапрос

Ссылается на столбцы внешнего запроса. Логически выполняется для каждой внешней строки.

Найти товары дороже средней цены своей категории

```
SELECT p.product_id, p.name, p.category, p.price
FROM products p
WHERE p.price > (
     SELECT AVG(p2.price) FROM products p2
     WHERE p2.category = p.category
);
```

Для каждого клиента посчитать число его заказов отдельным подзапросом

```
SELECT c.customer_id, (
     SELECT COUNT(*)
     FROM orders o
     WHERE o.customer_id = customer_id
) AS orders_cnt
```

```
FROM customers c
ORDER BY orders_cnt DESC, c.customer_id;
```

Ловушка NOT IN и NULL

Если подзапрос возвращает хотя бы 1 `NULL`, то `NOT IN` никогда не вернет `TRUE`.

Операции над множествами

Операции над множествами объединяют результаты по вертикали и добавляют строки

Операция	Результат
`UNION`	Все строки без дублей
`UNION ALL`	Все строки + дубли
`INTERSECT`	Строки, которые есть в обоих подзапросах
`EXCEPT`	Строки первого подзапроса, которых нет во втором

Правила операций

1. Число стоблцов в обоих запросах совпадает, типы совместимы
2. Имена столбцов берутся из первого запроса
3. Операции без `ALL` удаляют дубли
4. `ORDER BY` и `LIMIT` пишутся 1 раз в конце и относится ко всему результату.
5. Два `NULL` считаются одинаковыми
6. `INTERSECT` приоритетнее, чем `UNION` и `EXCEPT`
7. `EXCEPT` не симметричен

Примеры

Города, где есть и клиенты, и курьеры

```
SELECT city FROM customers
INTERSECT
```

```
SELECT city FROM couriers
ORDER BY city;
```

Города, где есть клиенты, но нет курьеров

```
SELECT city FROM customers
EXCEPT
SELECT city FROM couriers
ORDER BY city;
```